Very very very Hidden

⚠️ **Security Warning**: Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

📋 Challenge Description

Finding a flag may take many steps, but if you look diligently it won't be long until you find the light at the end of the tunnel. Just remember, sometimes you find the hidden treasure, but sometimes you find only a hidden map to the treasure.

💡 Hints

- I believe you found something, but are there any more subtle hints as random queries?
- The flag will only be found once you reverse the hidden message.

🧰 Tools Used

- `file` - Determine file type
- `strings` - Extract printable character sequences
- `exiftool` - Read metadata
- `binwalk` - Detect embedded files
- `xxd` - View hexadecimal representation
- **Wireshark** - Packet analysis and HTTP object export
- **Python** - Custom steganography decoder (Invoke-PSImage)
- **PIL/Pillow** - Python imaging library

🔍 Complete Solution

Step 1: Download and Prepare

Download the PCAP file named `try_me.pcap` from the challenge page.

Step 2: Basic File Inspection

```
file try_me.pcap
```

Expected output:

```
try_me.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4
(Ethernet, capture length 262144)
```

The file is a standard **PCAP** capture containing network traffic.

Step 3: Search for Visible Text

```
strings try_me.pcap | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible.

Step 4: Inspect Metadata

```
exiftool try_me.pcap
```

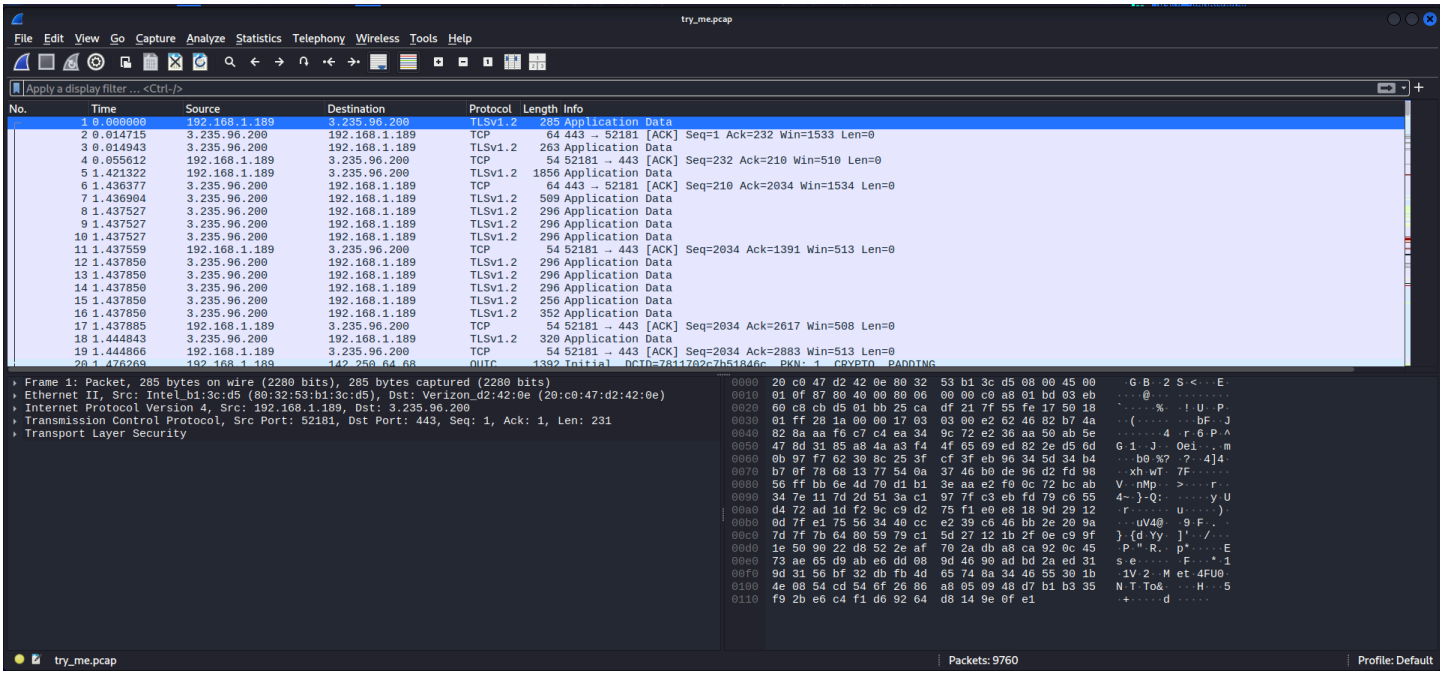
Output:

```
ExifTool Version Number         : 13.50
File Name                       : try_me.pcap
Directory                       : .
File Size                       : 9.7 MB
...
File Type                       : PCAP
Link Type                       : BSD Loopback
Time Stamp                      : 2021:01:25 01:55:43.944021+02:00
```

Nothing suspicious.

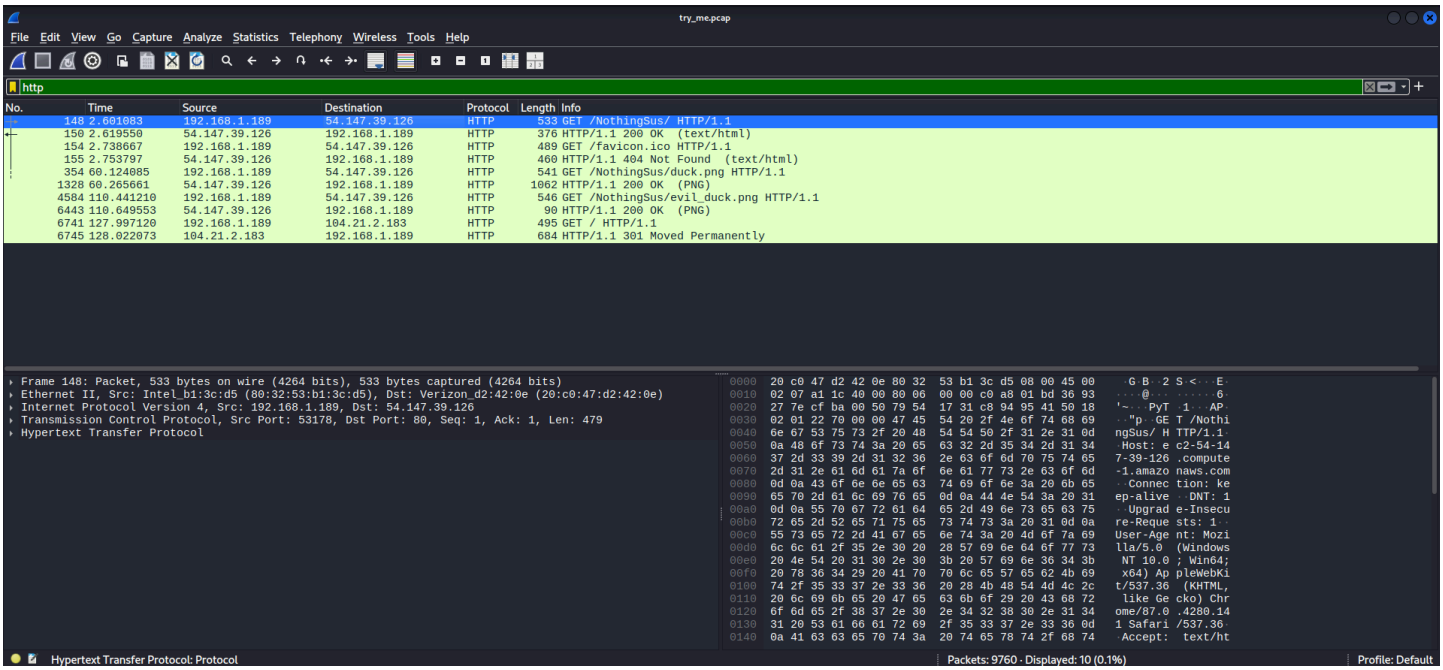
Step 5: Open the PCAP in Wireshark

```
wireshark try_me.pcap
```



Step 6: Filter HTTP Traffic

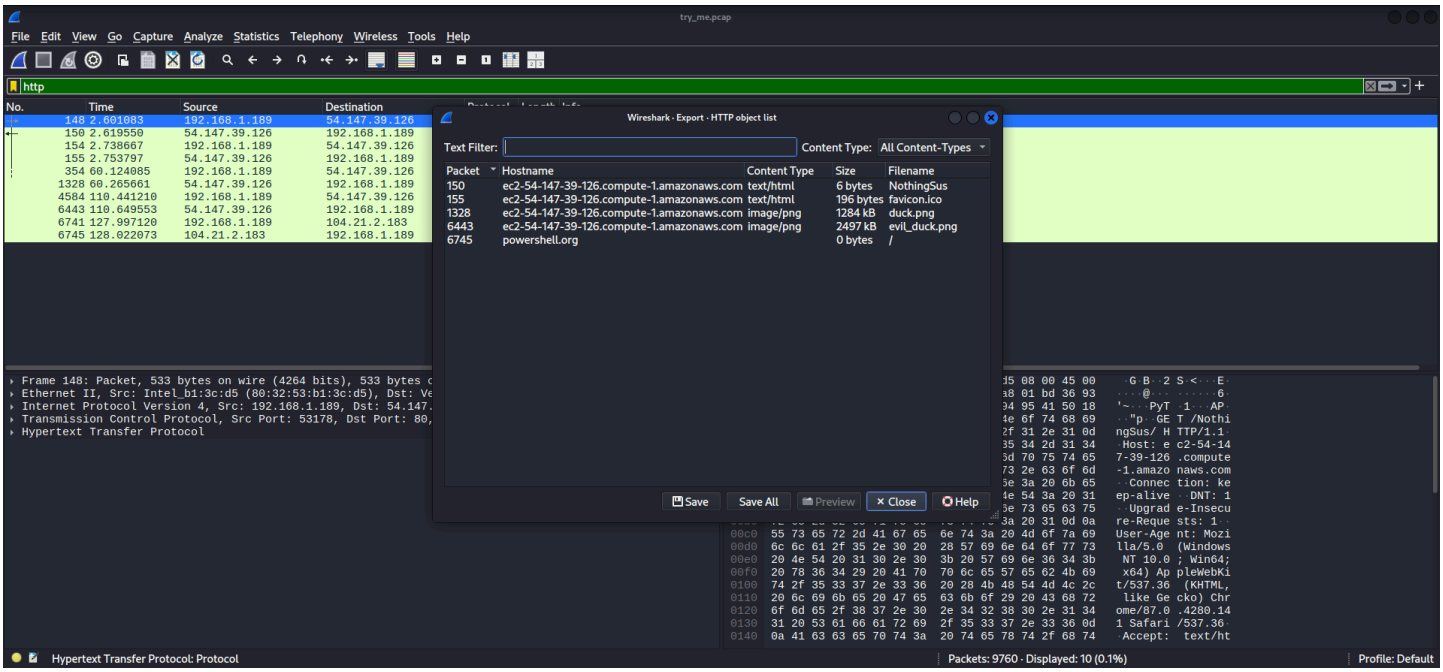
Apply the filter `http` to view only HTTP packets:



Step 7: Export HTTP Objects

Wireshark can extract files transferred over HTTP:

1. Go to **File → Export Objects → HTTP**



2. Several files appear in the list, including two PNG images
3. Save both images:
   - `duck.png`
   - `evil_duck.png`

Step 8: Examine the Extracted Images

```
file duck.png evil_duck.png
```

Output:

```
duck.png:      PNG image data, 1223 x 812, 8-bit/color RGB, non-interlaced
evil_duck.png: PNG image data, 1223 x 812, 8-bit/color RGB, non-interlaced
```

Both images have the same dimensions. `duck.png` appears normal, but `evil_duck.png` is suspicious (the name suggests malice).

Step 9: Search for Strings in the Images

```
strings evil_duck.png | grep -i "picoCTF"
```

Output: (no results)

The flag isn't directly visible.

Step 10: Scan with Binwalk

```
binwalk evil_duck.png
```

Output:

```
DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             PNG image, 1223 x 812, 8-bit/color RGB, non-interlaced
2862          0xB2E           Zlib compressed data, compressed
```

No obvious embedded files.

Step 11: Examine with Exiftool

```
exiftool evil_duck.png
```

The output shows standard PNG metadata (ICC profile, gamma, etc.), but nothing immediately suspicious.

However, noticing the HTTP traffic contained PowerShell references, this suggests the image may have been generated by **Invoke-PSImage** - a PowerShell tool that hides scripts in image pixels.

Step 12: Understand Invoke-PSImage Steganography

Invoke-PSImage is a PowerShell steganography tool that:

1. Encodes a PowerShell script into the **least significant bits (LSB)** of image pixels
2. Uses the lower 4 bits of the **Green** and **Blue** channels to store data
3. The output is a valid PNG image that visually looks normal

Decoding formula (per pixel):

```
char = ((Blue & 0x0F) << 4) | (Green & 0x0F)
```

The extracted bits form a PowerShell script containing two strings that XOR to reveal the flag.

Step 13: Python Decoder Script

python

```python
#!/usr/bin/env python3
from PIL import Image
import sys
import re

def decode_invoke_psimage(image_path):
    """
    Decodes PowerShell steganography payloads created by Invoke-PSImage.
    The payload is stored in the least significant 4 bits of the Green and Blue
channels.
    """
    try:
        img = Image.open(image_path).convert('RGB')
        pixels = img.load()
        width, height = img.size
    except Exception as e:
        print(f"Error opening image: {e}", file=sys.stderr)
        return

    payload_chars = []

    # Iterate through pixels row by row, left to right
    for y in range(height):
        for x in range(width):
            r, g, b = pixels[x, y]

            # Extract lower 4 bits of Blue (high nibble of char)
            # Extract lower 4 bits of Green (low nibble of char)
            char_code = ((b & 0x0F) << 4) | (g & 0x0F)

            # Check if it's a valid ASCII character
            if 32 <= char_code <= 126 or char_code in [10, 13, 9]:
                payload_chars.append(chr(char_code))
            else:
                # Stop on non-printable characters (end of payload)
                break
```

```
            else:
                continue
            break

    payload = "".join(payload_chars)

    # Parse the PowerShell script to find string1 and string2
    # The output typically contains:
    # $string1 = "..."
    # $string2 = "..."

    string1_match = re.search(r'\$string1\s*=\s*"([^"]*)"', payload)
    string2_match = re.search(r'\$string2\s*=\s*"([^"]*)"', payload)

    # Alternative variable names (s1, s2)
    if not string1_match:
        string1_match = re.search(r'\$s1\s*=\s*"([^"]*)"', payload)
    if not string2_match:
        string2_match = re.search(r'\$s2\s*=\s*"([^"]*)"', payload)

    if string1_match and string2_match:
        s1 = string1_match.group(1)
        s2 = string2_match.group(1)
        print(f"String1: {s1}")
        print(f"String2: {s2}")

        # XOR to get the flag
        flag = ""
        for i in range(len(s1)):
            flag += chr(ord(s1[i]) ^ ord(s2[i]))
        print(f"Flag: {flag}")
    else:
        print("Could not parse PowerShell script from extracted payload.")
        print(f"Extracted Payload Preview:\n{payload[:200]}")

if __name__ == "__main__":
    if len(sys.argv) > 1:
        decode_invoke_psimage(sys.argv[1])
    else:
        print("Usage: python decode_stego.py <image.png>")
```

Step 14: Run the Decoder

```
python decode_stego.py evil_duck.png
```

Output:

```
String1: <REDACTED>
String2: <REDACTED>
Flag: picoCTF{<REDACTED>}
```

Step 15: Alternative - Using tshark to Extract Images

Instead of manually exporting from Wireshark GUI, use `tshark`:

```
# Extract all HTTP objects
tshark -r try_me.pcap --export-objects "http,./extracted"

# List extracted files
ls extracted/
```

Then decode the extracted `evil_duck.png` as above.

✅ Final Result

```
picoCTF{<REDACTED>}
```

> **Note**: The actual flag is redacted here. In the real challenge, running the decoder script will reveal the complete flag.

🧠 Key Concepts

Concept	Description
HTTP Object Export	Wireshark feature to extract files transferred over HTTP.

Concept	Description
Invoke-PSImage	PowerShell tool for hiding scripts in PNG images using LSB steganography.
LSB Steganography	Hiding data in the least significant bits of image color channels.
4-bit LSB	Using the lower 4 bits of Green and Blue channels to store one character per pixel.
XOR Encoding	The hidden PowerShell script contains two strings that XOR to form the flag.
Multi-Layer Hiding	The flag is hidden in a PNG that was transferred over HTTP in a PCAP.

💡 Pro Tips

1. **Always export HTTP objects** - Transferred files often contain hidden data or steganography.
2. `evil_duck.png` **is a clue** - The name suggests malicious content (Invoke-PSImage).
3. **Invoke-PSImage is a known tool** - Recognizing the tool helps you find the correct decoder.
4. **The decoder looks for** `$string1` **and** `$string2` - Some versions use `$s1` and `$s2` instead.
5. **Heuristic stopping** - The script stops when it hits non-printable characters to avoid trailing noise.
6. **XOR requires equal length strings** - Both strings should be the same length; the result is the flag.

🔗 Alternative Decoding Tools

Tool	Purpose
Invoke-PSImage	The original tool (can also decode)

Tool	Purpose
Python script (above)	Custom decoder for this challenge
StegSolve	GUI tool for bit-plane analysis

🏁 Conclusion

The PCAP file `try_me.pcap` contained HTTP traffic. By exporting HTTP objects in Wireshark, we discovered two PNG images: `duck.png` and `evil_duck.png`. The name `evil_duck.png` suggested malicious steganography.

Recognizing that `evil_duck.png` was created with **Invoke-PSImage**, we wrote a Python decoder to extract the hidden payload from the lower 4 bits of the Green and Blue channels. The extracted payload was a PowerShell script containing two strings (`$string1` and `$string2`) that XORed to reveal the flag.

This challenge demonstrates:

- **Network forensics** - Extracting transferred files from PCAPs
- **Steganography detection** - Recognizing LSB-based steganography
- **Invoke-PSImage** - A real-world PowerShell steganography tool
- **Multi-layer decoding** - PCAP → HTTP → PNG → LSB → PowerShell → XOR → Flag

Key takeaway: When you see an image named “evil” or similar, suspect steganography. Tools like Invoke-PSImage hide scripts in image pixels using LSB, and the same technique can be reversed to extract hidden payloads. Always examine HTTP objects in PCAPs - they often contain the next layer of the puzzle.